

Predicción del Valor de Mercado de Autos Usados

Documentación Técnica y Estructura del Proyecto — Rusty Bargain

1. Descripción del Proyecto

Este proyecto ha sido desarrollado para **Rusty Bargain**, un servicio de venta de vehículos de segunda mano en pleno proceso de creación de una aplicación móvil corporativa. La finalidad de esta herramienta es permitir a los usuarios calcular de forma instantánea y fidedigna el valor de mercado de sus vehículos basándose en su historial, equipamiento y especificaciones técnicas.

Para garantizar una óptima experiencia de usuario (UX) y viabilidad operativa en arquitecturas móviles, el modelo de Machine Learning no solo debe ser preciso, sino que debe cumplir con estrictos criterios de eficiencia técnica definidos por el negocio.

Criterios clave de evaluación para Rusty Bargain:

- **Calidad de la predicción:** Evaluada estrictamente mediante la métrica RECM (Raíz del Error Cuadrático Medio).
- **Velocidad de la predicción:** Tiempo de latencia mínimo al procesar consultas individuales en la app.
- **Tiempo de entrenamiento:** Costo temporal y computacional requerido para reentrenar el modelo periódicamente.

2. Estructura de los Datos

El dataset de origen se ubica en la ruta `/datasets/car_data.csv` y se compone de las siguientes variables informativas:

Característica	Descripción
DateCrawled	Fecha en la que se descargó el perfil de la base de datos.
VehicleType	Tipo de carrocería del vehículo (Categoría).
RegistrationYear	Año de matriculación del vehículo.
Gearbox	Tipo de caja de cambios (Manual/Automático).
Power	Potencia del motor en caballos de vapor (CV).
Model	Modelo específico del vehículo.
Mileage	Kilometraje acumulado (según métricas regionales).
RegistrationMonth	Mes de matriculación del vehículo.
FuelType	Tipo de combustible consumido.
Brand	Marca fabricante del coche.
NotRepaired	Estatus de reparación previa del vehículo (Sí/No).
DateCreated	Fecha de creación del perfil en la plataforma.
NumberOfPictures	Número de fotografías adjuntas en la publicación.
PostalCode	Código postal del propietario.
LastSeen	Fecha de la última actividad del usuario en el servicio.
Price (Target)	Precio del vehículo en euros (€). Variable objetivo del modelo.

3. Arquitectura y Modelado Técnico

El desarrollo requiere la comparativa de múltiples familias de algoritmos y configuraciones de hiperparámetros. Las directrices metodológicas incluyen:

- **Prueba de Cordura (Sanity Check):** Implementación inicial de un modelo de *Regresión Lineal*. Sirve como línea base fundamental; cualquier algoritmo avanzado de Gradient Boosting debe superar obligatoriamente este puntaje.
- **Modelos Basados en Árboles:** Evaluación de *Árboles de Decisión* y *Bosques Aleatorios (Random Forest)*, aplicando sintonización fina de hiperparámetros.

- **Potenciación del Gradiente (Gradient Boosting):** Aprendizaje autónomo y despliegue de la librería **LightGBM**. Adicionalmente, se recomienda contrastar con **CatBoost** y **XGBoost** bajo esquemas parametrizados.

Codificación de Características Categóricas

Es crítico observar la naturaleza de cada algoritmo para la ingeniería de características. Mientras que los modelos lineales y **XGBoost** exigen una transformación explícita como *One-Hot Encoding (OHE)*, librerías como **LightGBM** y **CatBoost** poseen optimizaciones internas nativas de alta eficiencia para procesar variables de tipo categórico sin expandir la dimensionalidad de la matriz de datos de forma innecesaria.

4. Instrucciones de Implementación y Monitoreo

Para cuantificar de manera exacta los tiempos de ejecución solicitados por el negocio, se debe utilizar el comando mágico de celda en entornos Jupyter Notebook antes de iniciar el bloque de entrenamiento o predicción:

```
%%time
# Código de entrenamiento del modelo para calcular latencia exacta
model.fit(features_train, target_train)
```

En caso de experimentar saturación de memoria RAM del kernel de Jupyter debido al volumen de datos tras realizar codificaciones OHE masivas, se aconseja liberar espacio explícitamente mediante el operador nativo de Python:

```
del features_train
del features_valid
```

5. Stack Tecnológico Requerido

- **Lenguaje:** Python 3.x
- **Procesamiento de Datos:** Pandas, NumPy
- **Modelado y Métricas:** Scikit-Learn (Métrica RECM / RMSE)
- **Frameworks de Boosting:** LightGBM, CatBoost, XGBoost